

+

×

—

÷

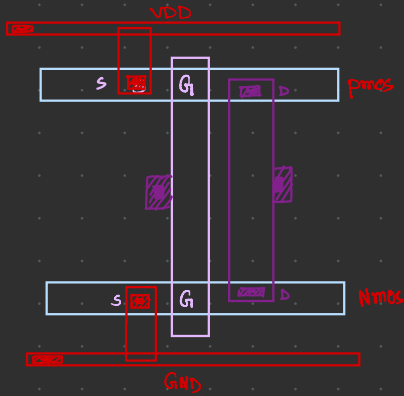
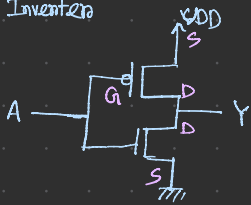
+

×

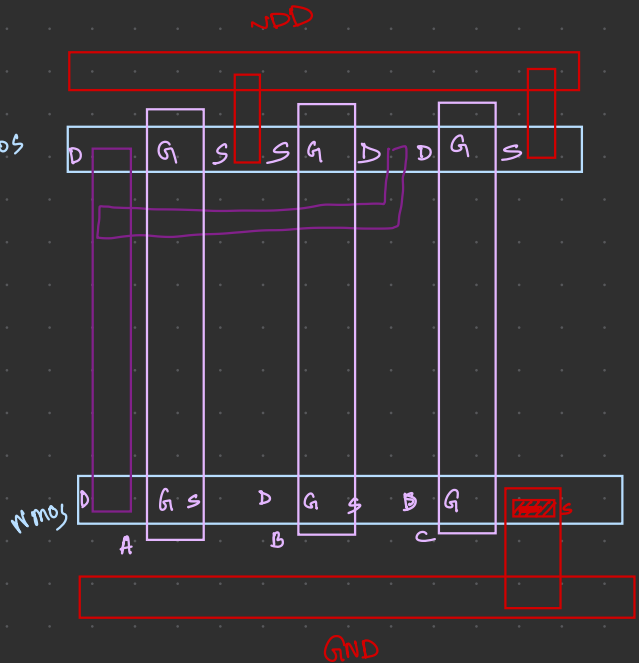
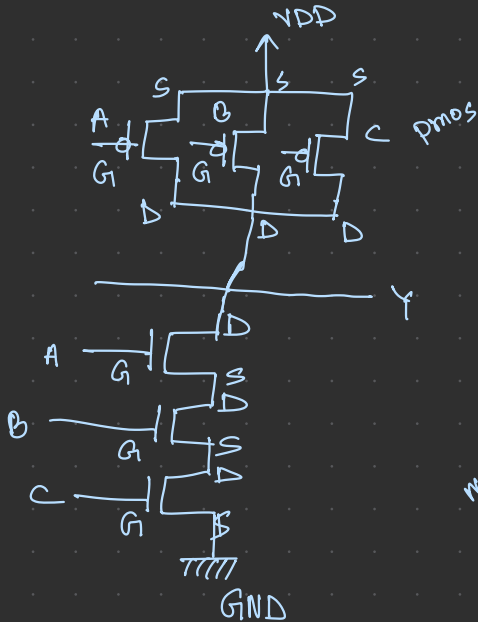
—

÷

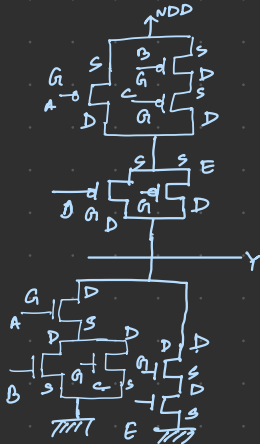
Invertier



$$Y = \overline{A \cdot B \cdot C}$$

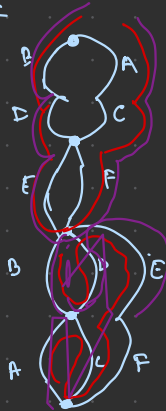
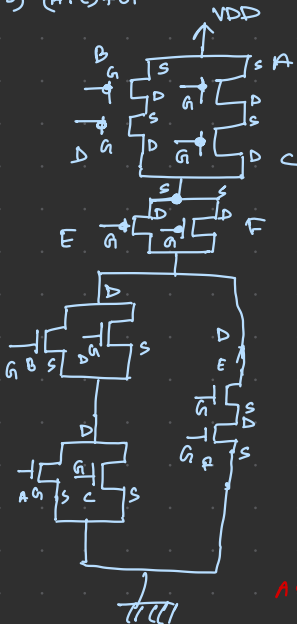


$$Y = A(B+C) + DE$$



CBDE

$$Y = (B+D)(A+C) + EF$$



ACFEDB

EDBACF

	S	G	DS	G	DS	G	DD	G	SD	G	SD	G	S
A			C		F		E		D		B		
	S	G	DD	G	SS	G	DS	G	DD	G	SD	G	S
A			C		F		E		D		B		

CMOS Design Rule

λ (lambda) is a scaling parameter used in λ based design rules to simplify layout design and make it process dependent.

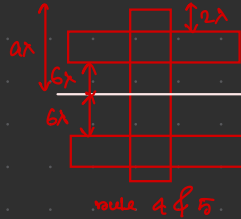
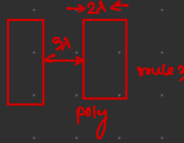
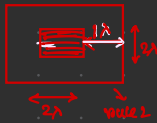
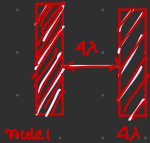
λ = one-half of the minimum feature size

Ex. for 180 nm $\lambda = 0.09 \mu\text{m}$

Advantages $\rightarrow$ Same layout can be scaled to different process just by changing λ .

Rules

1. Metal and diffusion have minimum width & spacing of 4λ .



2. contacts are $2\lambda \times 2\lambda$ and surrounded by 1λ on the layers above & below and spacing 3λ

3. polysilicon overlaps diffusion by 2λ where a transistor is desired & has a spacing of 1λ where no transistor is desired

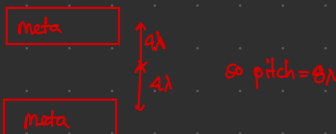
3. n-well surrounds PMOS by 6λ & avoids nmos transistor by 6λ

Area Estimation

$\rightarrow$ A wiring track is the space required for a wire.

Track pitch = Metal width + Metal spacing.

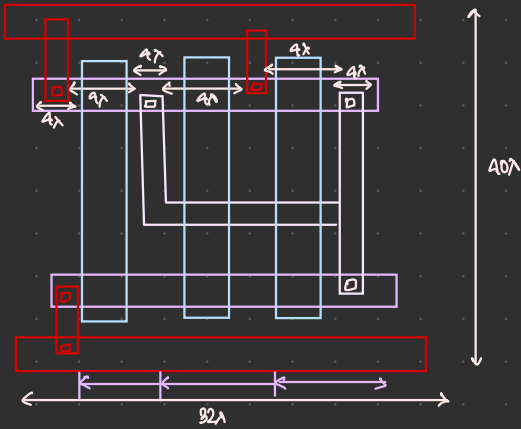
It defines how tightly interconnects can be placed in a layout.



Estimate Area by counting required metal wiring tracks.

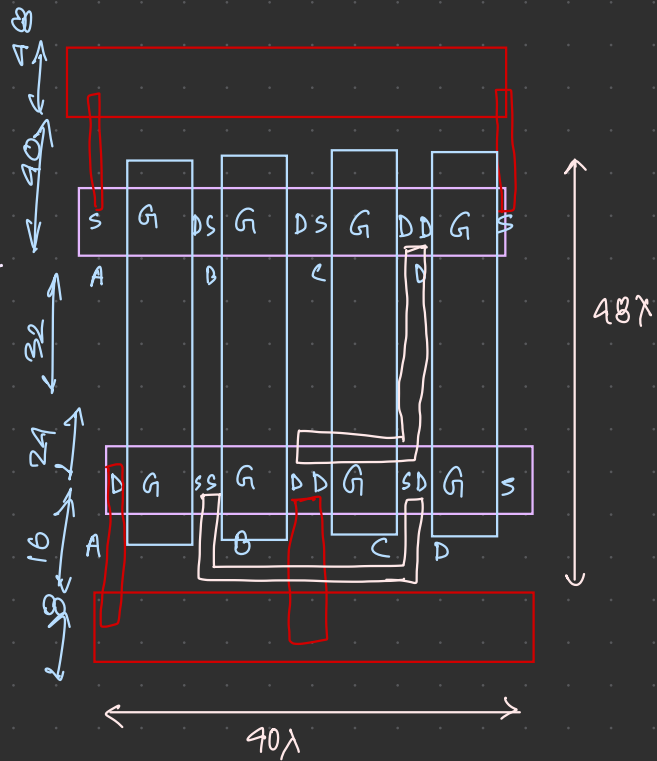
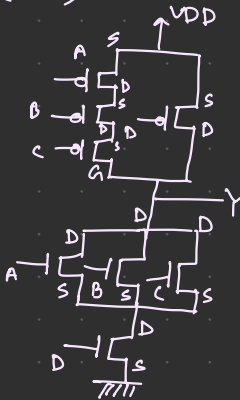
Multiply by 8 to express in λ .

3 input NAND



Area : $40\lambda \times 32\lambda$

$$Y = \overline{(A+B+C)D}$$

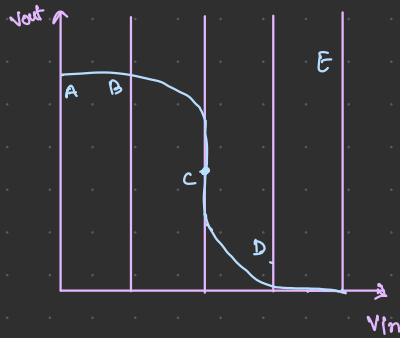
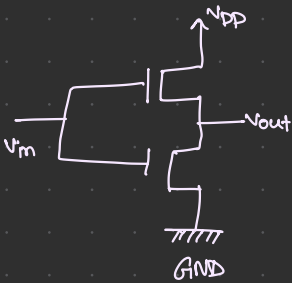


Dc Transfer characteristics

Dc curve $\longrightarrow$ V_{out} vs V_{in}

Re-writing eqn in terms of V_{out} & V_{in}

	cut-off	Linear	Saturation
NMOS	$V_{gsn} < V_{th}$	$V_{gsn} > V_{th}$	$V_{gsn} > V_{th}$
	$V_{in} < V_{th}$	$V_{in} > V_{th}$	$V_{in} > V_{th}$
	$V_{out} < V_{in} - V_{th}$	$V_{out} > V_{in} - V_{th}$	$V_{out} > V_{in} - V_{th}$
PMOS	$V_{gsp} < V_{tp}$	$V_{gsp} > V_{tp}$	$V_{gsp} > V_{tp}$
	$V_{in} < V_{tp} + V_{DD}$	$V_{in} > V_{tp} + V_{DD}$	$V_{in} > V_{tp} + V_{DD}$
	$V_{out} < V_{in} - V_{tp}$	$V_{out} > V_{in} - V_{tp}$	$V_{out} > V_{in} - V_{tp}$



Regions	Conditions	p device	n device	Output
A	$0 < V_{in} < V_{th}$	Linear	cut-off	$V_{out} = V_{DD}$
B	$V_{th} < V_{in} < \frac{V_{DD}}{2}$	Linear	Saturation	$V_{out} > \frac{V_{DD}}{2}$
C	$V_{in} = \frac{V_{DD}}{2}$	Saturated	Saturation	V_{out} drops sharply
D	$\frac{V_{DD}}{2} < V_{in} < V_{DD} - V_{tp} $	Saturated	Linear	$V_{out} < \frac{V_{DD}}{2}$
E	$V_{in} > V_{DD} - V_{tp} $	cut-off	Linear	$V_{out} = 0$

Noise margin

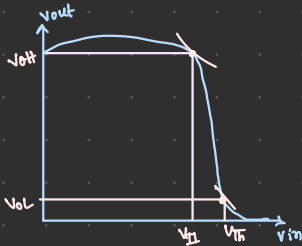
This parameter allows you to determine the allowable noise voltage on the input of a gate so that the output will not be corrupted.

$$NM_L = V_{IL} - V_{OL}$$

Low Noise margin

$$NM_H = V_{OH} - V_{IH}$$

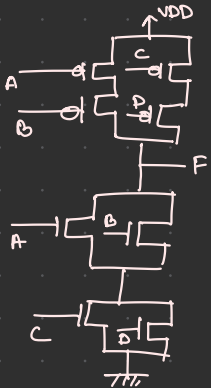
High Noise margin



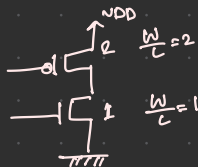
Transistor Sizing

Worst case: when there is at least one conducting path between V_{DD} to output for pull up or from output to GND for PDN. In worst case, one path will be considered at a time.

$$F = \frac{(A+B)(C+D)}{C+D}$$



Unit CMOS



$$\mu_p \left(\frac{W}{L}\right)_p = \mu_n \left(\frac{W}{L}\right)_n$$

$$\mu_n = 2\mu_p$$

$$2W_p = W_n$$

AC/AD/BC/BD

$$\rightarrow R_{eq, AC} = R_A + R_C$$

$$\Rightarrow \frac{1}{\left(\frac{W}{L}\right)_{AC}} = \frac{1}{\left(\frac{W}{L}\right)_A} + \frac{1}{\left(\frac{W}{L}\right)_C}$$

$$\Rightarrow 1 = \frac{2}{\left(\frac{W}{L}\right)_{A=C}}$$

$$\left(\frac{W}{L}\right)_{AC} = 2$$

$$R = \rho \frac{L}{A}$$

$$= \rho \frac{L}{Wt}$$

$$R \propto \frac{1}{W}$$

Pull-up: AB/CD

$$R_{eq} = R_A + R_B$$

$$\Rightarrow \frac{1}{\left(\frac{W}{L}\right)_{eq}} = \frac{1}{\left(\frac{W}{L}\right)_A} + \frac{1}{\left(\frac{W}{L}\right)_B} \Rightarrow \frac{1}{2} = \frac{2}{\left(\frac{W}{L}\right)_{A=B}}$$

$$\left(\frac{W}{L}\right)_{A=B} = 4$$

$$F = \overline{AB + C}$$



pull-down: AB/C

$$R_{eq} = R_A + R_B$$

$$\Rightarrow \frac{1}{\left(\frac{W}{L}\right)_{eq}} = \frac{1}{\left(\frac{W}{L}\right)_A} + \frac{1}{\left(\frac{W}{L}\right)_B}$$

$$\left(\frac{W}{L}\right)_{A-B} = 2$$

pull-up: A-c/B-c

$$R_{eq} = R_A + R_C$$

$$R_{eq} = R_c$$

$$\Rightarrow \frac{1}{\left(\frac{W}{L}\right)_{eq}} = \frac{1}{\left(\frac{W}{L}\right)_A} + \frac{1}{\left(\frac{W}{L}\right)_C}$$

$$\Rightarrow \frac{1}{\left(\frac{W}{L}\right)_{eq}} = \frac{1}{\left(\frac{W}{L}\right)_c}$$

$$\Rightarrow \frac{1}{2} = \frac{2}{\left(\frac{W}{L}\right)_{A-C}}$$

$$\left(\frac{W}{L}\right)_L = 1$$

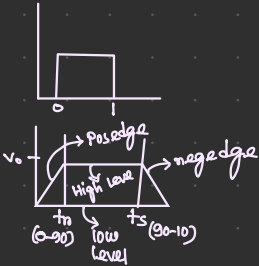
$$\left(\frac{W}{L}\right)_{AC} = \left(\frac{W}{L}\right)_{BC} = 4$$

Verilog Lec1

Net type connection: transfer signal one to another

Register type: Store the value.

event control



Assignments
{

 → continuous
 → procedural

#10 → 10 sec delay

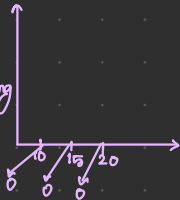
$$\begin{aligned} A &= 1 \\ B &= 0 \\ C &= 0 \end{aligned}$$

#10 reg1 = 1'60

#5 $C = \text{neg}!$

#5 $\text{reg}2 = B$

→ blocking Assign.



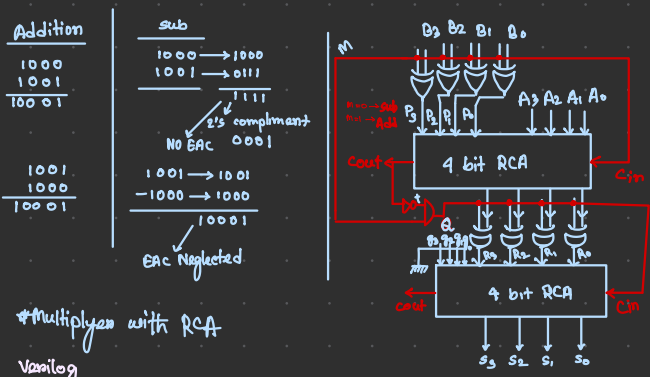
No blocking assignments $\Leftarrow$

Mid-Term
upto Verilog combinational circuit
 ↳ DLD book

- Question Pattern
- Math based
 - no theory
 - Verilog (no behavioural code allowed)

Wkasami
 → 20B, 30B, 2B3 (DLD-combinational ckt), verilog (until mid)

4 bit binary Adder/subtractor



*Multiplies with RCA

Verilog

Module four-bit-AS (S[3:0], Cout, M, Cin, A[3:0], B[3:0])
 output S[3:0], Cout
 Input M, Cin, A[3:0], B[3:0];
 wires q[3:0], p[3:0], r[3:0], n[3:0]

Cout, r, q
 XOR x0 (p[3], M, B[3])
 XOR x1 (p[2], M, B[2])
 XOR x4 (p[1], M, B[1])
 XOR x0 (p[0], M, B[0])
 not (t, Cout)
 and (a, t, M)
 Xor x03 (n[3], a, q[3])
 Xor x02 (n[2], a, q[2])
 Xor x01 (n[1], a, q[1])
 Xor x00 (n[0], a, q[0])

assign q3=0
 assign q2=0
 assign q1=0
 assign q0=0

four-bit-RCA M1 (Cout, q, [3:0], A[3:0], p[3:0], M);
 four-bit-RCA M2 (Cout', S[3:0], r[3:0], q[3:0], a);
 end module

$S_3S_2S_1S_0 = 2's \text{ complement when } S_0B \text{ with NO EAC}$

$q_3q_2q_1q_0 = S_3S_2S_1S_0$
 ↳ when adder + sub with RAC

M	Cout	Q
0	0	0
0	1	0
1	0	1
1	1	0

$Q = M \overline{Cout}$

Comparator

$$A = A_0 \quad B = B_0$$

$$\text{outcomes: } \begin{array}{c} A=B \\ E_0 \\ A_0 \end{array} \quad \begin{array}{c} A>B \\ A_0 \\ L_0 \end{array} \quad \begin{array}{c} A<B \\ L_0 \end{array}$$

if $A_0 = 0 \quad B_0 = 0 \quad E_0 = 1$
 $A_0 = 1 \quad B_0 = 1 \quad E_0 = 1$
 $A_0 = 1 \quad B_0 = 0 \rightarrow G_0 = 1$
 $A_0 = 0 \quad B_0 = 1 \rightarrow L_0 = 1$



verilog: Module one-bit-comp (E_0, A_0, L_0, A_0, B_0)

Output E_0, G_0, L_0

Input A_0, B_0

wire w_1, w_2

xor $G_1 (E_0, A_0, B_0)$

not $G_2 (w_1, B_0)$

not $G_3 (w_2, B_0)$

2 bit comparator

$$A = A_1 A_0 \quad B = B_1 B_0$$

$$A = B: \text{ if } A_0 = B_0 \text{ \& } A_1 = B_1 \quad E = E_1 E_0$$

$$E_0 = A_0 B_0 \quad E_1 = A_1 B_1$$

$$A > B$$

$$\text{if i) } A_1 > B_1 \text{ or}$$

$$\text{ii) if } A_1 = B_1 \text{ \& } A_0 > B_0$$

$$G_2 = G_1 + E_1 \cdot G_0$$

$$A < B$$

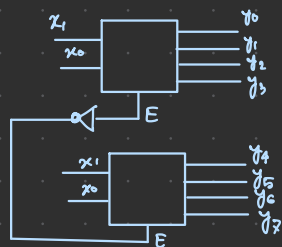
$$A < B_1 \text{ or } A_1 = B_1 \text{ \& } A_0 < B_0$$

$$L_2 = L_1 + E_1 \cdot L_0$$

2-bit comparator verilog.

Exam: 4/16-bit comp.

* 3x8 decoder using 2x4 decoder



* Step 1: design 2 to 4 bit decoder

* Step 2: use 2 to 4 bit to design 3 to 8 bit decoder

Truth Table

x_2	x_1	x_0	y_3	y_6	y_5	y_4	y_7	y_2	y_1
0	0	0							1
0	0	1						1	
0	1	0						1	
0	1	1				1			
1	0	0			1				
1	0	1		1					
1	1	0	1						
1	1	1							

* 4x2 priority encoder: $D_0 > D_2 > D_3 > D_1$

E	D_3	D_2	D_1	D_0	y_1	y_0
1	x	x	x	1	0	0
1	x	1	x	0	0	1
1	1	0	x	0	1	0
1	0	0	1	0	1	1
0	0	0	0	0	0	0

code

module encoders (d, y);

input [3:0] d;

output reg [1:0] y;

output reg v;

always @ (*) begin

case x (d)

4'b xxx1: begin

y = 2'b00;

v = 1;

end

4'b x1x0: begin

y = 2'b01;

v = 1;

end

4'b 10x0: begin

y = 2'b10;

v = 1;

end

4'b 0010: begin

y = 2'b11;

v = 1;

end

default: begin

y = 2'b00;

v = 0;

endcase

end

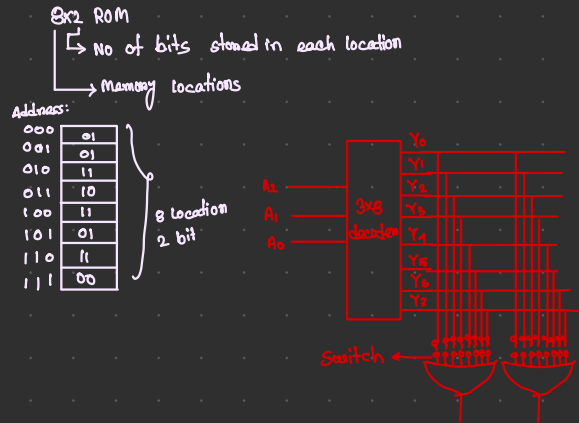
endmodule.

1-2:1 MUX

2-4:1 MUX

3-4:1 MUX using 2:1 MUX

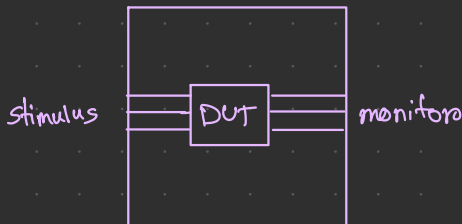
Read only Memory:



verilog Test Bench

Test Bench:

- A verilog procedural block that executes only once.
- used only for simulation
- Generates clock, reset & required input/output



- create a dummmy template
- Declare inputs to the DUT as 'reg' & output as wire
- Instantiate DUT
- Initialization & Monitoring
 - #assign some values to the DUT
 - #monitor the DUT outputs for functional

Example:

module example (A, B, C, D, E, F, Y);

input A, B, C, D, E, F;

output Y;

wire t1, t2, t3, Y;

NAND g1 (t1, A, B);

AND g2 (t2, ~B, D);

NOR g3 (t3, E, F);

NAND g4 (Y, t1, t2, t3);

end module

module test_bench

.A(A), .B(B), .C(C)

neg A, B, C, D, E, F

wire Y;

example DUT (A, B, C, D, E, F, Y)

initial
begin

\$monitor (\$time "A=%b, B=%b; C=%b; D=%b, E=%b, F=%b, Y=%b", A, B, C, D, E, F, Y)

#10 A=1; B=0; C=0; D=1, E=0; F=0;

#5 A=0; C=1;

#5 finish

end module

Simulator, Directives

\$display ("<format>" exp1, exp2,)

-used to print the immediate values of text on variables.

\$monitor ("format" var1, var2, ...)

-similar to display but does not print immediately.

\$display ("%b %d %h", a, b, c)

\$monitor ("%d %b %b", \$time, a, b)

\$finish -Terminates the simulation process.

\$dumpfile (<filename>);

-specifies the file that will be used for storing the values of selected variables.

\$dump off

-stops the dumping

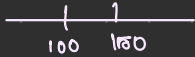
\$dump on

- starts previously stopped dumping

Ex.

initial

#100 \$dump off;



initial

#150 dump on;

\$dumpvars (level, list_of_variables/module)

specifies which variables should be dumped to the .vcd

\$dumpall

- the current values of all.

variables will be written to the file.

\$dumplimit (file size)

- used to set the max size of vcd file.

\$dumpvars (0, module, module2)

\$dumpvars (1, a, b, c, d)

Flip Flop

SR FlipFlop

S	R	Q(next)
0	0	Q (prev) hold
0	1	0
1	0	1
1	1	Invalid

```
Verilog: module SR_ff (clk, S, R, Q)
    input clk, S, R;
    out reg Q;
    always @(posedge clk) begin
        case (fsup)
            2'b10: Q <= 1;
            2'b01: Q <= 0;
            2'b00: Q <= Q;
            2'b11: Q <= 1'bX // Invalid
        endcase
    end
endmodule
```

Test bench:

```
module test-bench;
    reg clk, S, R;
    wire Q;
    SR_ff DUT (.clk(clk), .S(S), .R(R), .Q(Q));
    initial begin
        clk = 0;
        forever #10 clk = ~clk;
    end
    initial begin
        $monitor ("%t| %b %b %b| %b, %time, clk, S, R, Q");
        S = 0; R = 0; #10
        S = 0; R = 1; #10
        S = 1; R = 0; #10
        $finish;
    end
endmodule
```



J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

```

VHDL
module JK_FF(Q2,K,clk)
output reg Q
input S,K,ck
always @ (posedge clk)
begin
    case C{S,K}
    2'b00: Q<=Q
    2'b01: Q<=0
    2'b10: Q<=1
    2'b11: Q<=!Q
    end
end
endmodule

```

Q_n	Q_{n+1}	S	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

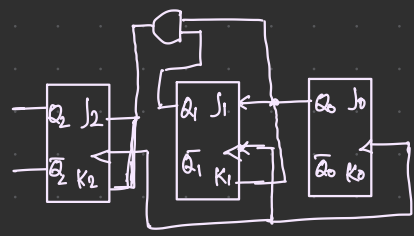
end
endmodule

3-bit / countdown

000 → 001 → 010 → 011 → 100 → 101 → 110 → 111

present state	next state	FF input functions		
$Q_2 Q_1 Q_0$	$Q_2 Q_1 Q_0$	$J_2 K_2$	$J_1 K_1$	$J_0 K_0$
000	001	0 X	0 X	1 X
001	010	0 X	1 X	X 1
010	011	0 X	X 0	1 X
011	100	1 X	X 1	X 1
100	101	X 1	0 X	1 X
101	110	0 X	1 X	X 1
110	111	X 0	X 0	1 X

$J_0 = K_0 = 1$
 $J_1 = \sum m(1, 3, 5, 7)$
 $= Q_0$
 $K_1 = Q_0$
 $J_2 = K_2 = Q_1 Q_0$

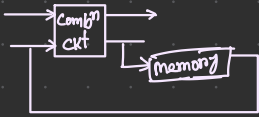


Verilog

module syn_counter (output [2:0] q, input [2:0] k, q, input clk

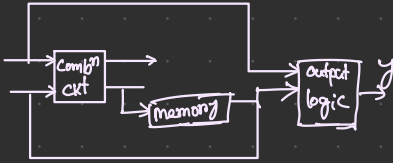
Finite state machine

seqⁿ ckt:



FSM $\rightarrow$ relay
FSM $\rightarrow$ Moore

Moore machine:



Output logic $\rightarrow$ two inputs

(i) input

(ii) present state of memory

output $Y = f(X, Q)$

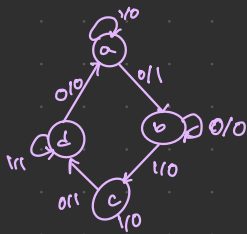
$\downarrow$
present state

Next state $Q^+ = g(X, Q)$

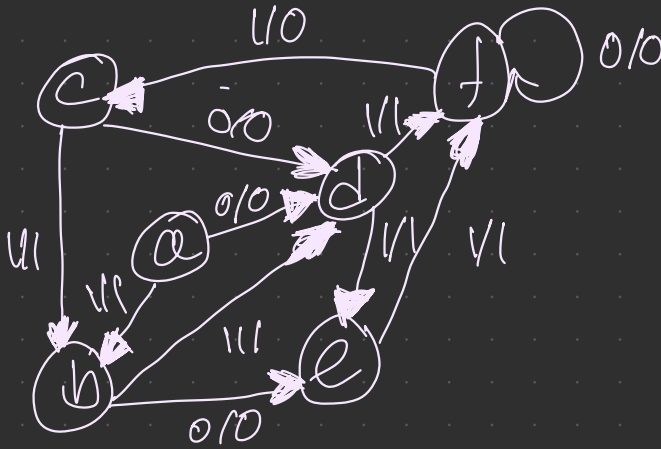
Example:



Finite state machine



State Reduction



Quiz-23 Nov → FSM

Quiz-04 Nov

*Design an FSM for the given state diagram:



present state	Next state		output	
	x=0	x=1	x=0	x=1
a	a	b	0	0
b	c	b	0	1
c	c	a	1	1

No of state = 3

flip-flop required = 2

a → 00

b → 01

c → 10

	Q_1	Q_0	x	Q_1^+	Q_0^+	D_1	D_0	Y
a	0	0	0	0	0	0	0	0
	0	0	1	0	1	0	1	0
b	0	1	0	1	0	1	0	0
	0	1	1	0	1	0	1	1
c	1	0	0	1	0	1	0	1
	1	0	1	0	0	0	0	1

D flip flop

$$Q_1^+ = D_1$$

$$Q_0^+ = D_0$$

K-map

$$D_1 = Q_0\bar{x} + Q_1x = Q_0 + Q_1$$

$$D_0 = \bar{Q}_1x \quad || \quad Y = \overline{Q_1 + Q_0x}$$

$$D_1 = f(Q_1, Q_0, x) = \sum m(2, 4) + \sum d(6, 8)$$

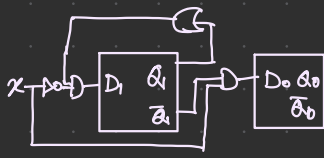
$$D_0 = g(Q_1, Q_0, x) = \sum m(1, 3) + \sum d(6, 8)$$

$$Y = h(Q_1, Q_0, x) = \sum m(3, 5) + \sum d(6, 8)$$

$$\begin{aligned} D_0 &= \bar{Q}_1\bar{Q}_0x + \bar{Q}_1Q_0x \\ &= \bar{Q}_1x(Q_0 + \bar{Q}_0) \\ &= \bar{Q}_1x \end{aligned}$$

$$\begin{aligned} D_0 &= \bar{Q}_1\bar{Q}_0x + \bar{Q}_1Q_0x \\ &= \bar{Q}_1x(Q_0 + \bar{Q}_0) \\ &= \bar{Q}_1x \end{aligned}$$

$$\begin{aligned} D_1 &= \bar{Q}_1\bar{Q}_0\bar{x} + \bar{Q}_1Q_0\bar{x} \\ &= \bar{Q}_1\bar{x}(\bar{Q}_0 + Q_0) \\ &= \bar{Q}_1\bar{x} \end{aligned}$$



Verilog

Module Dflipflop (D, clk, Q)

Input D, clk;

output Q;

always @ (posedge clk)

begin
Q <= D;

end
endmodule

module fsm (Y, x, clk)

output reg y;

input x, clk;

wire w1, w2;

Not (w2, x);

* Design a FSM for the given state diagram.

$\frac{2}{0}$

$\frac{b}{a}$

$\frac{2}{0}$

$\frac{0}{0}$

$\frac{d}{a}$

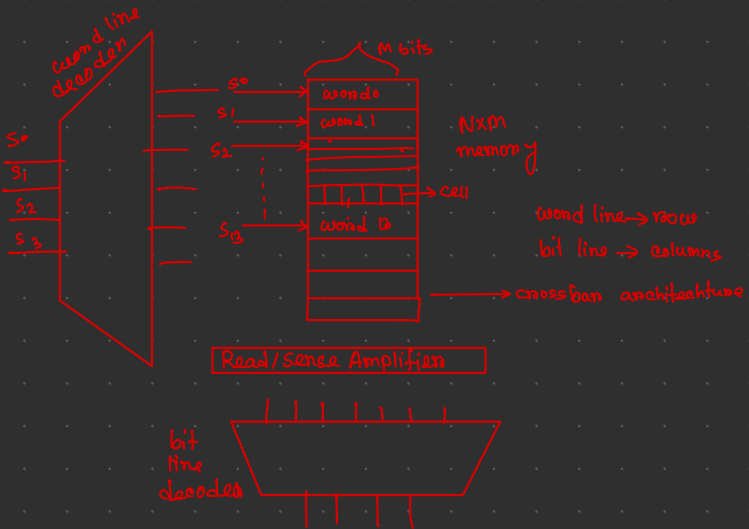
→ memory theory

Memory

Non-Volatile - doesn't depend on power → ROM

Volatile → depends on power → RAM

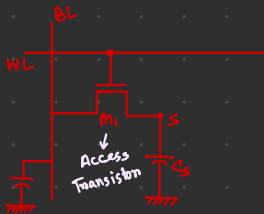
Array based memory system



SRAM - Static Random Access memory → minimum six transistor

DRAM - Dynamic Random Access memory → simple design
↳ Refresh

1-T1 DRAM



Write: Transistor → called access transistor

WL: Driving the gate of transistor.

If $WL = V_{DD}$ → transistor on, it is replaced by short-ckt. the capacitor is connected to the BL.

Let us assume C_s Precharged to V_{DD} .

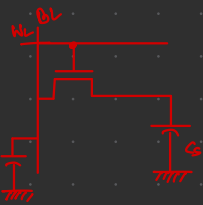
Make $BL = 0$, C_s will discharged all its voltage to BL & it will turn to zero.

(1) so we successfully converted 1 to 0 → write 0.

Now, C_s = Precharged to zero

(0) Apply $BL = V_{DD}$, C_s will change to V_{DD}

(1) Zero converted to 1 → write 1.



Read: for read operation, $C_L = \frac{V_{DD}}{A}$

Let $C_S = V_{DD}$

Now,

Node 2 voltage $\rightarrow V_{DD}$

Node 1 voltage $\rightarrow \frac{V_{DD}}{2}$



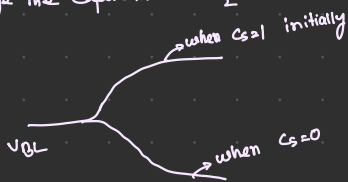
capacitor will change the
BL from $\frac{V_{DD}}{2}$

Let's
 $C_S = 0$ (storing 0)

BL will change the capacitor to $\frac{V_{DD}}{2}$

Let
 $C_S = 0$ (storing 0)

BL will change the capacitor to $\frac{V_{DD}}{2}$



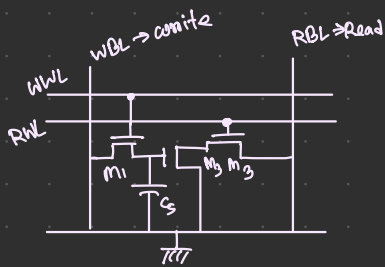
We use a sense amplifier to read the bit stored in C_S



when $V_{BL} > V_{ref} \rightarrow \text{logic 1 (read } C_S = 1)$

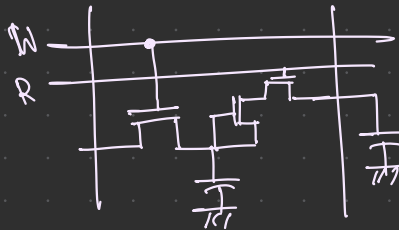
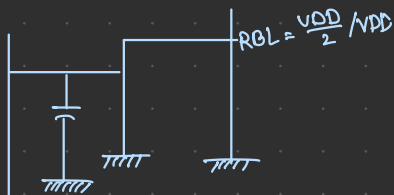
$V_{BL} < V_{ref} \rightarrow \text{logic 0 (read } C_S = 0)$

3-Transistor D-RAM



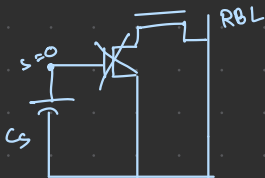
Read: RBL precharged to $\frac{V_{DD}}{2}$ on V_{DD} . Let C_s node storing V_{DD} ($C_s = 1$ on V_{DD})

M_2 turns on WBL



Let, C_s storing 0 ($C_s = 0$)

M_2 off

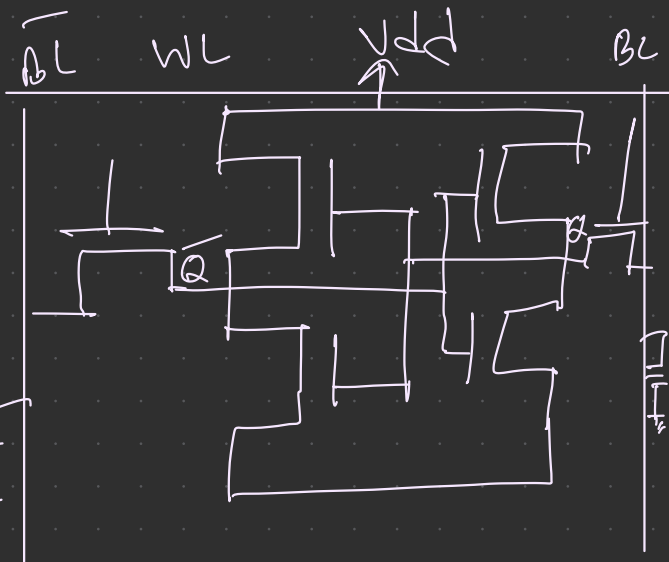
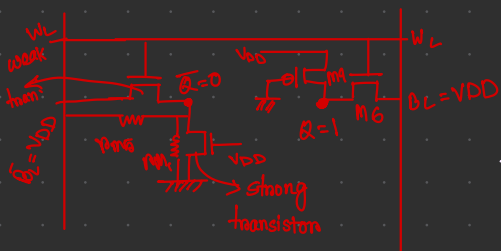
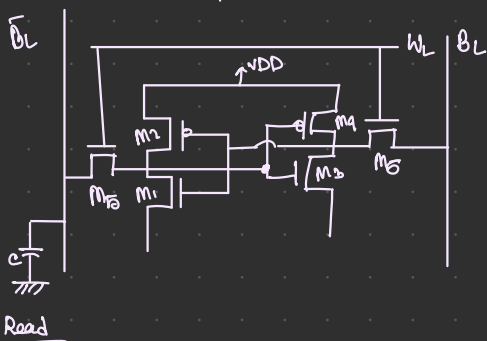
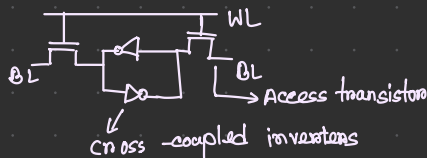


RBL voltage same.

RBL $\rightarrow$ discharging $\rightarrow$ read stored 1 in C_s

RBL $\rightarrow$ not discharging $\rightarrow$ read stored 0 in C_s
on same

6-T SRAM Cell



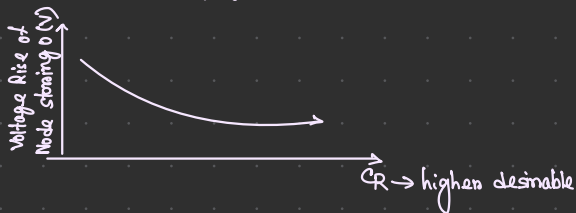
Precharge B_L & $\bar{B}_L$ to V_{DD}

$$W_L = V_{DD}$$

$$V_Q = \frac{R_{M1} V_{DD}}{R_{M1} + R_{M5}}$$

$$V_Q \downarrow, n_m \downarrow, \frac{W_1}{L_1} \uparrow, n_{m5} \uparrow, \frac{W_5}{L_5} \downarrow$$

$$\text{Cell ratio} = \frac{W_1/L_1}{W_5/L_5} \approx \frac{W_1}{W_5}$$



$$W_L = 1$$

$$\bar{Q} = 1, Q = 0$$

precharged to V_{DD}

B_L voltage drop

$Q/p_1 \leftarrow$ Sense amp

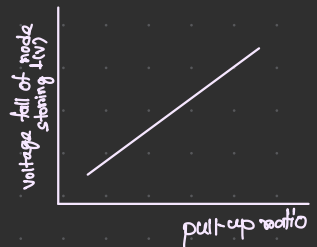
change $0 \rightarrow 1$ & $1 \rightarrow 0$

Apply opposite voltage to corresponding BL node.
(vdd, 0)

$L = V_{DD}$

$\begin{matrix} p m \downarrow & W \uparrow \\ r m \uparrow & W \downarrow \end{matrix}$

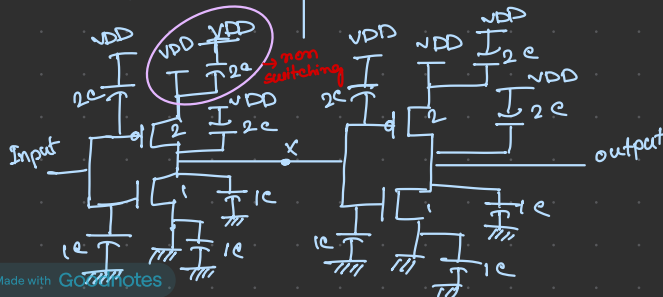
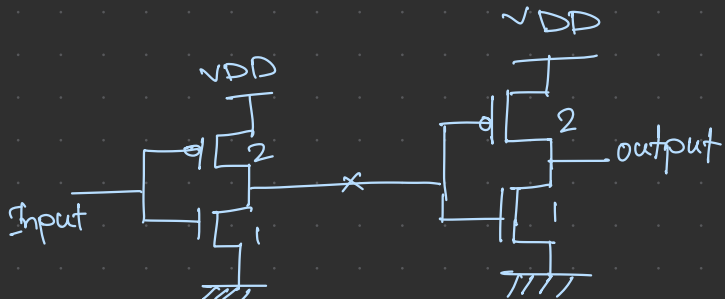
$$\text{pull-up ratio (PR)} = \frac{W_A/L_A}{W_B/L_B} = \frac{W_A}{W_B}$$



1. Hold stability

1. Hold stability $\longrightarrow$ Reduce access transistor width
2. Read stability $\longrightarrow$ Increase pull down transistor
3. Write ability/write speed $\longrightarrow$ Increase access transistor width
4. Access time/read speed $\longrightarrow$ Reduce pull-up transistor width
5. Cell leakage $\longrightarrow$ Increase access transistor width
6. Cell area $\longrightarrow$ Reduce pull-down transistor width
7. Read write power $\longrightarrow$ Minimum size transistor

The diagrams illustrate the transformation of a MOSFET circuit into a Norton equivalent circuit. The top diagram shows a MOSFET with gate voltage g , drain node d , and source node s . This is transformed into a circuit with a dependent current source βV_{gs} in parallel with a resistor R_{DS} and a capacitor C_K . The bottom diagram shows a MOSFET with gate voltage g and drain node d . This is transformed into a circuit with a dependent current source βV_{gs} in parallel with a resistor R_{DS} and a capacitor C_K . The source node s is connected to ground through a resistor R_K and a capacitor C_K .



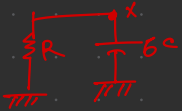
$\hookrightarrow (2C+1C) + (2C+1C) = 6C$
 ↓ ↓
 1st stage 2nd stage
 output input

$t_{pdf} \Rightarrow$ propagation delay falling

$t_{pdr} \Rightarrow$ propagation delay rising

t_{pdf}

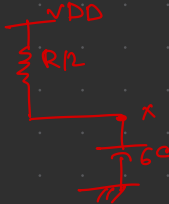
Input to $\rightarrow$ node x



$$t_{pdf} = RC$$

t_{pdr}

Input to $\rightarrow$ node x



for, $x \rightarrow$ output

$$t_{pdf} = RC$$

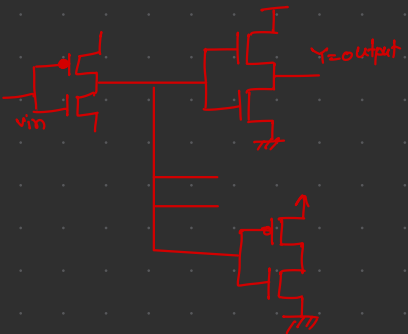
$$t_{pdr} = \frac{C}{2} RC$$

$$t_{pdf} = t_{pdf} + t_{pdr}$$

i/p $\rightarrow$ x o/p $\rightarrow$ x

$$= RC + \frac{RC}{2} = \frac{3RC}{2}$$

unit inverter driving m inverters



Inv of ∞ scaled unit driving m numbers

$$t_{pdf} = \frac{R}{W} 3WC + \frac{R}{W} 3mC$$

$$= 3RC \left(1 + \frac{m}{W}\right)$$

$$\frac{m}{W} = h = \text{fanout/electrical effort}$$

$$= \frac{\text{Load cap}}{\text{output cap}} = \frac{C_{ext}}{C_{in}}$$

$$t_{pdf} = 3RC + (3C)mR = 3RC(1+m)$$

$V_{in} \rightarrow x$

$$t_{pdf} = 3RC(1+m) + 3RC$$

$V_{in} \rightarrow y$

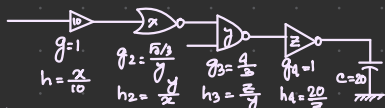
$$t_{pd} = 3RC(1+h)$$

$$h = \frac{3Cm}{3CW} = \frac{m}{W}$$

Elmore delay

$$C_p = \sum_i R_{n-i} \quad C_i = \sum_{a=1}^n C_i \sum_{u=1}^L R_j$$

Delay in multistage program



find x, y, z to minimize delay.

path logical effort, $G = \prod g_i = (1) \left(\frac{\sqrt{5}}{2}\right) \left(\frac{4}{3}\right) (1) = \frac{20}{9}$

path Electrical effort, $H = \frac{C_{out}(path)}{C_{in}(path)} = \frac{20}{10} = 2$

path effort delay, $f = gh$ $P + gh$ $\rightarrow f$

path delay

$$D \leq (f + P_i)$$

$$= \sum f_i + \sum P_i$$

$$= DP + P$$

P is constant regardless of sizing

let us assume all the gates having same size

$$\therefore f_i = \hat{F} = g_i h_i = F^{1/N}$$

N = number of logic gates

The minimum delay $D = NF^{1/N} + \sum P_i$

now $\theta = 1$

$$\hat{F} = F^{1/N} = \left(\frac{40}{2}\right)^{1/4} = 1.45$$

$$h_4 = \frac{C_{out}}{C_{in4}} = \frac{20}{2}$$

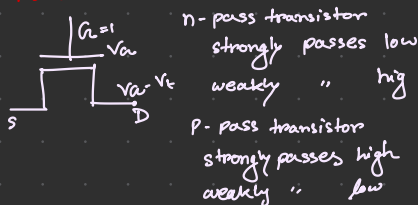
$$h_4 = \frac{1.45}{g_4} = \frac{20}{2}$$

$$z = \frac{20(1)}{1.45} = 13.8$$

$$h_3 = \frac{z}{g_3} = \frac{1.45}{g_3}; y = \frac{4.3 \times 2.8}{1.45} = 12.7$$

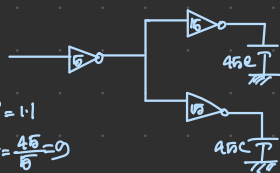
$$h_2 = \frac{y}{2} \Rightarrow x = 14.6$$

Pass Transistor



Designing AND gate using n-pass & p-pass chosen as control unit

A	B	AB=Y
0	0	0
0	1	0
1	0	0
1	1	1



$$G = 1.1$$

$$H = \frac{4.5}{5} = 0.9$$

$$F = \prod f_i = \left(\frac{1.5+1.5}{5} \cdot 1\right) \left(\frac{1.5}{1.5} \cdot 1\right) = 1.8$$

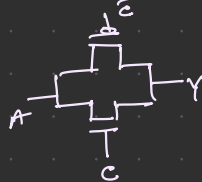
$$\text{Branching effort} = \frac{C_{on-Path} + C_{off-Path}}{C_{on path}} = 2$$

$$\text{path Branching effort } \theta = \prod b_i$$

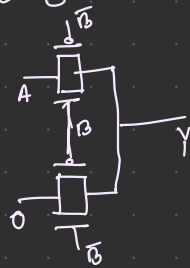
$$\text{path effort } F = \prod f_i = G \theta H$$

AND, OR, XOR, XNOR, NAND

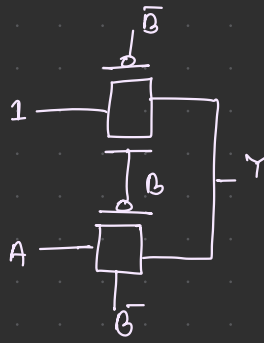
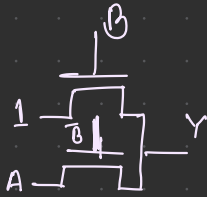
Transmission gate:



Designing AND,

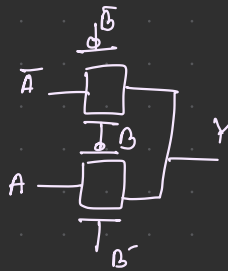


A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1



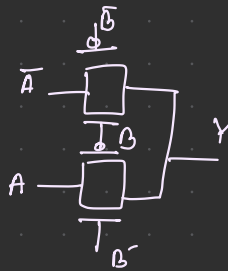
XOR

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0



XNOR

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1



NAND

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

